Week 3
2. Introduction to the UNIX File I/O System

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Wednesday, September 17th, 2025

COMP 2560 System Programming: Standard Input/Output Library

School of Computer Science
University of Windsor

Instructor: Dr. Andreas Maniatis

Content

Introduction

The Standard Input/Output Library was written by Dennis Ritchie around 1975.

This library is specified by the ANSI C standard because it has been implemented on different operating systems.

This library handles details such as

- buffer allocation
- performing I/O in optimal-sized chunks

The header file of this library is normally in /usr/include/stdio.h

Streams and FILE objects

Opening or creating a file → associating a stream with the file.

fopen()

The function `fopen()` returns a pointer to a **`FILE`** object. A `FILE` object is a structure that contains all needed information to manage a stream:

- the file descriptor: a nonnegative integer used for the actual I/O
- a pointer to a buffer for the stream
- the size of the buffer
- a count of the characters currently in the buffer
- an error flag
- an end-of-file flag

Normally, an application software never needs to examine the `FILE` object.

Buffering

Goal of buffering

Minimize the number of I/O system calls.

There are three types of buffering provided:

- fully buffered,
- line buffered,
- unbuffered.

Fully buffered I/O

In the case of fully buffered I/O, actual I/O take place only when the I/O buffer is full.
Disk files are fully buffered by the standard I/O library.
The buffer is usually created using `malloc` the first time I/O is performed on a stream.

Flushing buffers

We can call the function `fflush()` to flush a stream forcing its associated buffer to be written even when it is partially filled.

Syntax:

```
int fflush(FILE *fp)
```

When `fp` is `NULL`, `fflush()` causes all output streams to be flushed.

Line buffered I/O

In this case of Line buffered I/O, actual I/O takes place only when a new line character is encountered on input or output.

Line buffering is typically used for the standard input and standard output streams.

Actual I/O will also take place when the line buffer is full before a new line is encountered.

Unbuffered I/O

In this case, the standard I/O library does not buffer the characters.
→ each time we print or read a single character, the actual I/O operation takes place.
For example, the standard error stream is normally unbuffered.

ISO C buffering requirements

ISO C requires the following buffering characteristics:

1. Standard I/O are fully buffered if and only if they do not refer to an interactive device.
2. Standard error is never fully buffered.

Changing the default buffering

We can change the default buffering using:

```
void setbuf(FILE *fp, char *buf);
```

Must be used **before** any read/write.

Used to substitute `buf` in place of the buffer normally allocated by the standard I/O library. The size required of `buf` is determined by the constant `BUFSIZ` in `stdio.h`

If `buf` is `NULL` then, buffering is disabled.

Changing the default buffering

Finer control on buffering

```
void setvbuf(FILE *fp, char *buf, int mode,
size_t size);
```

This function can specify which buffering we want depending on the value of `mode`:

- `_IOFBF`: fully buffered
- `_IOLBF`: line buffered
- `_IONBF`: unbuffered

When using `_IONBF`, the `buf` and `size` arguments are ignored.

Example 1

```
//ex1.c
#include <stdio.h>

#include <unistd.h>// This is needed for sleep()

int main(void){     // without fflush
  int i=0;
  char line[100] = "Hello, my name No-Name\n";
  while(line[i] != '\0'){
    putchar(line[i++]);
    sleep(1);
  }
}
```

Example 2

```
//ex2.c
#include <stdio.h>

#include <unistd.h>// This is needed for sleep()

int main(void){// forcing the flush with fflush
  int i=0;
  char line[100] = "Hello, my name No-Name\n";
  while(line[i] != NULL){
    putchar(line[i++]);
    fflush(stdout);   // flush std output buffer
    sleep(1);
  }
}
```

Example 3

```
 //ex3.c
#include <stdio.h>
#include <unistd.h>    // needed for sleep()

int main(int argc, char *argv[]){

  while(1){
   printf("Hello, program is runing right now");
   sleep(1);
  }
}
```

Example 4

```
//ex4.c
#include <stdio.h>
#include <unistd.h>    // needed for sleep()

int main(int argc, char *argv[]){

  while(1){
   printf("Hello, program is runing right now\n");
   sleep(1);
  }
}
```

Standard Input, Standard Output and Standard Error

```
#include <stdio.h>

int main(int argc, char *argv[]){
  FILE *fd;
  char c;

  if(argc==1)
    fd=stdin;
  else
    if((fd = fopen(argv[1], "r"))==NULL){
      fprintf(stderr, "Error opening %s, exiting\n", argv[1]);
      exit(0);
    }
  while( (c=getc(fd)) != EOF)
    putc(c, stdout);

  exit(0);
}
```

Opening a Stream

Three functions can be used to open a standard I/O stream:

`FILE *fopen(const char *f, const char *t)`
this is the most used one.

Example: `file = fopen(``./data.txt'', ``r'')`

`More details on slide 20 for fopen(…)`

Opening a Stream

Three functions can be used to open a standard I/O stream:

```
FILE *freopen(const char *f, const char *t,
              FILE *fp)
```

The `freopen` function opens a specified file on a specified stream, closing the stream first if it is already open.

This function is typically used to open a specified file as one of the predefined streams: standard input, standard output, or standard error.

```
if (freopen("new.input", "r", stdin)==NULL)…
```

Opening a Stream

Three functions can be used to open a standard I/O stream:

```
FILE *fdopen(int filedesc, const char *t)
```

This function associates a standard I/O stream with an existing file descriptor (the `filedesc` argument).

See **`freopen.c`** sample code

File descriptors

A file descriptor is a nonnegative integer used by the kernel to specify any open file.
A file descriptor is typically returned by the system call `open()` that opens/creates a file, a pipe or, a network communication channel.
The function `fopen()` cannot be used to open a pipe or a network communication channel

→ we use the system call *`open()`* to get a file descriptor for a pipe or a channel, then we use `fdopen()` to associate it with a standard stream.

Argument types

The argument type, `const char *t`, specifies how a stream is to be opened.

The possible values of type are:
r for read, **w** for write, **a** for append at the end of the file, **r+** for read and write, **w+** for read and write and, **a+** for read and write at the end of the file (append).

Summary

Restrictions	r	w	a	r+	w+	a+
File must exist already	*			*		
Previous contents of file lost		*			*	
Stream can be read	*			*	*	*
Stream can be written		*	*	*	*	*
Stream can be written only at the end			*			*

Restrictions

When a file is open for read and write, the following restrictions apply:

> Output (reading) cannot be directly followed by input (writing) without an intervening `fseek or rewind`.
> Input (writing) cannot be directly followed by output (reading) without an intervening `fseek or rewind` or, an input operation that encounters an EOF.

`int fclose(FILE *fp)`

`int fclose(FILE *fp)` closes any opened stream. In particular:

- Any buffered output data is flushed,
- Any buffered input data is discarded,
- Any allocated buffer is released.

Note that when a process terminates normally, all open standard I/O streams are closed.

Unformatted I/O

There are 3 types of unformatted I/O:

Single-character: I/O

Line I/O: to read or write a line at a time

Direct I/O: also called **binary I/O**. Useful when dealing with structures and binary information.

Single-character Input functions

`int` **`getc`**`(FILE *)`, `int` **`fgetc`**`(FILE *)` and `int` **`getchar`**`(void)`.
We have:

- `getchar(void)` is equivalent to `getc(stdin)`.
- `getc()` can be implemented as a macro whereas `fgetc()` cannot.
  - → `getc` is more efficient,
  - → the address of `fgetc` can be passed as a parameter unlike `getc` (macros do not have addresses).

Important issues about these functions

They return the next character as an **`unsigned char`** **converted into an `int`**. **Reason**: the high order bit is set without causing the return value to be negative.
→ we can read all possible 255 different values of a byte. In particular, we will never get a character with -1 (`EOF`) as its value.
→ The return value from these functions **can't** be stored in a `char` variable then compared to `EOF`

Example... (filesize1.c)

```
#include <stdio.h>

int main(int argc, char *argv[]){
  FILE *fp;
  char ch;
  int fileSize=-1;

  fd = fopen(argv[1], "r");  do{
    Ch = getc(fp);
    fileSize++;
  } while(ch != EOF);
      printf("Size of %s is %d\n", argv[1], fileSize);
}
```

`./size size.c` → Size of `size.c` is 516 (correct size)

...Example..(filesize2.c)

However, if the file (example.txt) you are reading containing special character.....

```
#include <stdio.h>

int main(int argc, char *argv[]){
  FILE *fd;
  char ch;
  int fileSize=-1;
  fd = fopen(argv[1], "r");
  do{
    ch=getc(fd);
    fileSize++;
  } while( ch != EOF);
  printf("Size of %s is %d\n", argv[1], fileSize);
}
```

`./size size.c` → Size of `size.c` is 6 (incorrect size, should be 23)

...Example (filesize3.c)

This can be solved simply by using `int` instead of `char`.

```
#include <stdio.h>

int main(int argc, char *argv[]){
  FILE *fd;
  int ch;
  int fileSize=-1;

  fd = fopen(argv[1], "r");  do{                    //ÿ
    Ch =getc(fd);
    fileSize++;
  } while( ch != EOF);

  printf("Size of %s is %d\n", argv[1], fileSize);
}
```

`./size size.c` → Size of `size.c` is 259 (correct size)

Note that these 3 functions return -1 whether an error or the end-of-file occurs.

How to differentiate between the 2 situations?

For each stream, two flags are maintained in the `FILE` object: an error flag and an end-of-file flag.
The following three functions should be used to access these flags:

`int ferror(FILE*)`
returns nonzero for true, 0 otherwise.

`int feof(FILE*)`
returns nonzero for true, 0 otherwise.

`void clearerr(FILE*)`
clears both the error and the end-of-file flags.

One more example:

`MyCopy.c`

Pushing back a character

`int ungetc(int c, FILE *fp)`

The function `int ungetc(int c, FILE *fp)`
Return `c` if OK, -1 otherwise.
This function is used to push back a character after reading from a stream.
This is useful when we need to peek at the next character to determine what to do next.

Note that:

- The pushed-back character does not have to be the same as the one that was read.
- We cannot push back *EOF*
- We can push back a character after end-of-file is reached. Then, we can read that character because *`ungetc()`* clears the end-of-file flag of the stream.

Single-character Output functions

`int` **`putc`**`(int c, FILE *fp)`: print a single character
`int` **`fputc`**`(int c, FILE *fp)`: same as `putc()` except that, `putc()` can be implemented as a macro whereas `fputc()` cannot.
`int` **`putchar`**`(int c)`: equivalent to `putc(c, stdout)`.

All these functions return the printed character `c` as an `int` when successful and -1 otherwise.

See sample code **getchar-putchar.c**

Example

```
//ex5.c, run it as ./a.out ex5.txt
#include <stdio.h>

int main(int argc, char *argv[]){
  FILE *f;
  char c;

  f=fopen(argv[1], "w");
  while((c=getchar()) != EOF)
    fputc(c, f);
}
```

Question: what happens if we exit with a CTRL-C?

Example

If this is a serious issue, then we should modify our code to:

```
#include <stdio.h>
int main(int argc, char *argv[]){  FILE *f;
  char c;

  f=fopen(argv[1], "w");
  setbuf(f, NULL);
  while((c=getchar()) != EOF)
       fputc(c, f);
}
```

or to

```
#include <stdio.h>
int main(int argc, char *argv[]){  FILE *f;
  char c;

  f=fopen(argv[1], "w");
  while((c=getchar()) != EOF){
    fputc(c, f);
       fflush(f);
  }
}
```

String/Line Input functions

Input functions:

`char *fgets(char *dest, int n, FILE *fp)`

Reads up through, including, the next newline, but no more than *n-1* characters. Then `NULL` is added to whatever has been read.

`char *gets(char *dest)`

Reads from `stdin` in a similar way as `fgets()` but has some *serious side effects*. In particular, it allows buffer to overflow. It should never be used!

Note that `dest` is the address of the array to read the characters into it. Both functions return `dest` if OK, `NULL` otherwise (end-of-file or error).

String/Line Output functions

Output functions:

`fputs``(const char *str, FILE *fp)`
Write the `NULL` terminated string, stored in `str`, to a stream.

`puts``(const char *str)`
Write the `NULL` terminated string, stored in `str`, to `stdout` and add a `newline` character at the end.

Note that `NULL` character at the end is not written by both functions.

See sample code `yesno.c`

Thank you!
Questions?

Dr. Andreas S. Maniatis
Assistant Professor
Andreas.Maniatis@UWindsor.ca
http://www.linkedin.com/in/andreasmaniatis